Installation manual: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

Karol Daniela Correa

November 8, 2026

Contents

Version Control 6
Update Rule 6
Introduction 7
Objective 8
Specific Objectives 8
Scope 8
Content Covered by the Manual 8
Content Not Covered by the Manual 9
Installer Profile 9
Required Knowledge 9
Installer Responsibilities 9
Limits of the Role 10
General System Description 10
Nature of the Installation 10
System Components 11
Network Organization 12
Prerequisites 13
Hardware Requirements 13
Operating System 14
Required Software 15
Optional Tools 15
Port Availability 16
Network Connectivity 17
Download and Installation 17
Installing Docker 17
Installing Git 18

Obtaining the Project 19
Verifying the Project Structure 19
Initial Configuration 20
Creating the Configuration File 21
Generating the Passwords and Keys 21
Environment Variables 22
Customization Cases 24
Email Channel 25
Bringing the System Up 25
Deploying the System 26
Startup Order 26
Database and Migrations 27
Initial System Data 28
Creating the First Administrator 28
Loading Test Data 29
Verifying the Installation 30
Checklist 30
First Test: Container Status 30
Second Test: Service Health 31
Third Test: Authentication 31
Fourth Test: Web Interface 32
Fifth Test: Network Isolation 32
Installing the Mobile Application 33
Additional Requirements 33
Installation Procedure 33
Day-to-Day Operation 34
Frequently Used Commands 34

Stopping and Restarting the System 34

Updating to a New Version 35

Uninstallation 35

Troubleshooting 35

Initial Diagnosis 35

Problems During Installation 36

Problems When Deploying the System 36

Problems with the Migrations 37

Connection and Usage Problems 38

Performance Problems 38

Clean Restart Procedure 39

Incident Escalation 39

Installation Without Containers 39

Tools Required by Programming Language 40

Recommended Approach: Hybrid Mode 40

Web Interface in Development Mode 40

Security Recommendations 41

Credentials 41

Network Exposure 41

Preservation of Information 41

Handover to the System Administrator 42

Glossary 42

References 43

Appendix A 44

Appendix B 45

Appendix C 45

Appendix D 46

Appendix E 47

Installation manual: FaceAttend EDU

Version Control

Every modification to this manual must be recorded before the document is submitted again. The record makes it possible to determine whether the manual corresponds to the version of the software being installed.

Table 1

Document version history

Version	Date	Description of change	Responsible
1.0	06/10/2026	Initial creation of the installation manual. Documents the deployment of the full set of 18 containers, configuration by means of the single environment file, the chain of versioned migrations, and the verification procedure.	Development team
1.1	06/10/2026	Application of the APA standards in their 7th edition and alignment with the common structure of the project manuals.	Development team
1.2	08/11/2026	Alignment with the delivered version: generation of passwords and keys, creation of the first administrator, gateway port 8000, project folder, and container count.	Development team

Source. Own elaboration.

Note. Minor changes, such as wording adjustments or the addition of screenshots, increment the second digit; changes that alter the installation procedure increment the first.

Update Rule

The manual must be reviewed after every deployment that modifies the services, the ports, the environment variables, or the versions of the required tools.

- Every addition of a new service to the container set requires updating the system description, the port map, and the verification procedure.
- Every new environment variable must be documented in the initial configuration section before submission.
- Every delivered version must be recorded in the table above together with its responsible party.

Introduction

FaceAttend EDU is a web and mobile platform intended for attendance management in educational institutions. The system automates check-in and check-out recording by means of facial recognition, retains a manual recording alternative, centralizes the attendance history, and manages the justification workflows, the role-differentiated reports, and the alerts addressed to the academic stakeholders.

This document explains, step by step, how to prepare the environment, configure the dependencies, and bring the system into operation on a local machine or on a server, starting from a machine with no prior installation.

The system does not constitute a single program, but rather a set of nine independent services, implemented in four different programming languages, which are deployed in a coordinated manner together with a gateway, three storage engines, and a messaging broker. For this reason, the installation procedure relies on containers, a mechanism that allows the entire set to be deployed with a single command.

All of the technical information set out here — ports, service names, configuration variables, paths, and initial credentials — corresponds to the configuration verified in the project repository.

Table 2 delimits the function of this document in relation to the other three manuals that make up the project documentation.

Table 2

Project manuals and the question each one answers

Document	Question it answers	Intended reader
User manual	How do I use the system?	End user
Technical manual	How is the system built?	Developer or support staff
Installation manual (the present document)	How do I install and run the system?	Technician responsible for bringing it up
Administrator manual	How do I administer the system?	User with the administrator role

Source. Own elaboration.

Objective

To enable a person outside the development team to install and bring the complete system into operation by following clear, complete, and verifiable instructions, without requiring additional assistance.

Specific Objectives

- State the hardware, operating system, and software requirements that the machine must meet before starting the procedure.
- Describe the installation of the base tools and the retrieval of the project source code.
- Explain the configuration of the system by means of the single environment variables file and document each available variable.
- Detail the deployment of the container set, its startup order, and the automated creation of the database.
- Provide a verification protocol that makes it possible to confirm, through concrete tests, that the installation is operational.
- Gather in a catalog the most frequent problems of the procedure together with their respective solutions.

Scope

Delimiting the scope prevents the reader from looking in this document for information that belongs in another of the project manuals.

Content Covered by the Manual

- Hardware, operating system, and software requirements.
- Installation of the base tools: the container platform and version control.
- Retrieval of the project source code.
- Configuration by means of environment variables.
- Deployment of the container set.
- Creation of the database and execution of the migrations.
- Functional verification of the installation.

- Resolution of the most frequent problems.
- Installation of the mobile application in development mode.

Content Not Covered by the Manual

- Functional use of the system, described in the user manual.
- The internal architecture and the source code, described in the technical manual.
- The management of users, roles, and reports, described in the administrator manual.
- Deployment in a production environment with a public domain, transport layer security certificates, and high availability.
- The configuration of continuous integration and continuous delivery.

Installer Profile

The document is addressed to the technician or the person responsible for bringing the system up: the instructor who deploys it for assessment purposes, the team member who sets up their workstation, or the systems administrator who installs it on an institutional server.

Required Knowledge

The reader is assumed to know how to open a terminal and run commands. No prior knowledge of containers, of microservice architectures, or of any of the programming languages used in the project is assumed: the necessary information is presented in the document itself and the technical terms are defined in the Glossary section.

Installer Responsibilities

- Verify that all of the prerequisites are met before downloading the project.
- Keep the configuration file outside the repository and replace the development credentials when the installation becomes accessible to third parties.
- Run the complete verification protocol before declaring the installation finished.
- Document, using the format in Appendix D, any incident that is not resolved by the Troubleshooting section.
- Hand over to the system administrator the initial credentials and warn them of the obligation to change them.

Limits of the Role

The installer does not administer the system once it is deployed. The creation of real users, the assignment of roles, the configuration of the academic parameters, and the generation of reports are the responsibility of the administrator and are described in their own manual. Neither does the installer modify the source code: when the procedure fails because of a software defect, their responsibility is to report it, not to fix it.

General System Description

It is worth understanding the nature of what is to be installed before starting the procedure.

Nature of the Installation

The system is built on a microservice architecture. Instead of a single large program, it is composed of nine independent services, each responsible for a delimited portion of the business: identity, authorization, academic management, scheduling, attendance, biometrics, configuration, notifications, and quality. In front of them operates a gateway that receives all of the requests and directs them to the corresponding service; behind them sit the databases and the messaging broker.

Installing this set manually would be unfeasible, since each service is implemented in a different programming language — Java, TypeScript, and Python — and has its own dependencies. For this reason, the project uses containers, a mechanism that packages each service together with all of its runtime requirements (Docker Inc., n.d.-a), and an orchestration tool that deploys the twenty-five containers in the correct order with a single command (Docker Inc., n.d.-b).

The practical consequence of this decision is relevant for the installer: there is no need to install Java, Node.js, or Python on the machine, since the container platform downloads and runs them inside each container. Only Docker and Git are required (Git, n.d.). Manual installation of the languages is described in the Installation Without Containers section and is only necessary when the source code is to be modified.

System Components

The complete deployment brings up twenty-five containers: sixteen that stay running and nine one-off tasks that prepare the database and then stop. They are grouped into three layers. Tables 3, 4, and 5 detail the components of the presentation layer, the application layer, and the data layer, respectively.

Table 3

Presentation layer components

Container	Technology	Port	Function
frontend-web	Expo Web and nginx	8090	Graphical interface loaded by the browser, built as a single-page application

Source. Own elaboration.

Table 4

Application layer components

Container	Technology	Port	Function
kong-gateway	Kong OSS 3.6	8000	Gateway: routes requests, applies the cross-origin policy and the rate limit
ms-identity	Java 21 and Spring Boot	8081	Persons, users, credentials, and sessions
ms-authorization	Java 21 and Spring Boot	8082	Roles and permissions
ms-academic	TypeScript and Fastify	8083	Campuses, programs, periods, cohorts, courses, and enrollments
ms-scheduling	Java 21 and Spring Boot	8084	Environments, schedule blocks, and class sessions
ms-attendance	Java 21 and Spring Boot	8085	Attendance records and justifications
ms-biometric	Python 3.12 and FastAPI	8086	Facial and fingerprint biometric vectors
ms-configuration	TypeScript and Fastify	8087	Academic and security parameters
ms-notification	TypeScript and Fastify	8088	Alerts and email sending
ms-quality	TypeScript and Fastify	8089	Quality assessment instrument

Container	Technology	Port	Function
redis	Redis 7	6379	Store for the gateway's rate limit

Source. Own elaboration.

Note. The nine microservices are identified by the prefix ms and are numbered 01 through 09 in the project repository.

Table 5

Data layer components

Container	Technology	Port	Function
postgres	PostgreSQL 17	5432	Main database, organized into eight schemas, one per bounded context
mongodb	MongoDB 7	27017	Storage of the biometric vectors
kafka	Kafka 3.8	9092	Messaging of domain events between services
minio	MinIO	Not published	Encrypted storage of the supporting documents of the justifications
Migrations (eight)	Liquibase 4.29	Not applicable	Temporary tasks that create the tables and load the initial data; they run, finish, and stop
db-service-logins	PostgreSQL 17	Not applicable	Temporary task that creates one database login per service, limited to its own schema

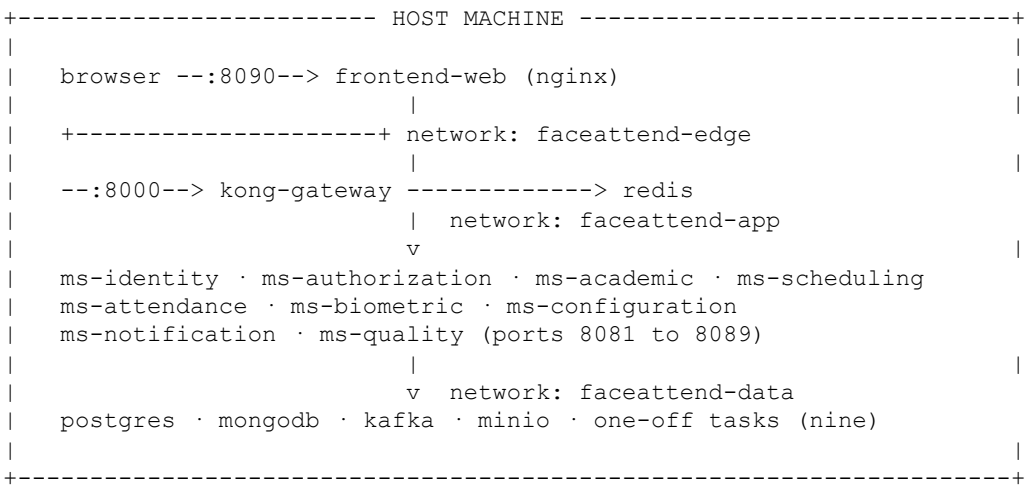
Source. Own elaboration.

Network Organization

The containers are distributed across three networks isolated from one another. The purpose of this separation is for the data layer to be unreachable from the network that serves the browser. Figure 1 represents this organization.

Figure 1

Deployment diagram and network separation



Source. Own elaboration.

Note. Only ports 8000, intended for the application programming interface, and 8090, intended for the web interface, remain open to the outside. The remainder listen exclusively on the loopback address 127.0.0.1, in accordance with the value of the BIND_IP variable.

For the installer, this organization means that, once the system is deployed, only two addresses are needed: http://localhost:8090 for the graphical interface and http://localhost:8000 for the application programming interface. The databases are not exposed and must not be opened to the outside.

Prerequisites

All of the requirements in this section must be verified before downloading the project. An unmet requirement causes failures that are difficult to diagnose at later stages of the procedure.

Hardware Requirements

The deployment keeps sixteen containers running at the same time, four of which run a Java virtual machine. Memory consumption is therefore considerable, and it is the requirement that is most frequently not met.

Table 6

Minimum and recommended hardware requirements

Resource	Minimum	Recommended	Remark
Random access memory	8 GB	16 GB	With 8 GB the system starts, although it is advisable to close the other applications. At least 6 GB must be allocated to the container platform
Processor	Four cores	Eight cores	Must support hardware virtualization and have it enabled in the basic input/output system
Disk space	20 GB free	30 GB free	Container images, source code, dependencies, and data volumes
Network connection	Internet access	Broadband	Needed only during the first build, to download approximately 5 GB

Source. Own elaboration.

Note. The memory values were estimated from the number of deployed containers and the characteristic consumption of the four Java virtual machines that make up the system.

Warning. The container platform does not work when hardware virtualization is disabled. On Windows, this condition is checked in Task Manager, Performance tab, CPU section, where it must read "Virtualization: Enabled". Otherwise, it must be enabled in the machine's basic input/output system, under the designations Intel VT-x, SVM Mode, or Virtualization Technology, depending on the manufacturer.

Operating System

Table 7

Supported operating systems

Operating system	Minimum version	Remark
Windows 10 and Windows 11	Build 19044, 64-bit	Requires the Windows Subsystem for Linux in its second version, which is installed together with the container platform
Ubuntu	22.04 LTS	Requires the container engine and the orchestration plugin. Debian 12 and Fedora 39 are equally supported
macOS	13 (Ventura)	Compatible with Intel processors and with Apple Silicon architecture processors

Source. Own elaboration.

Required Software

Only two tools are required, since the container platform provides the rest of the runtime environment.

Table 8

Software required for the installation

Tool	Minimum version	Source	Verification command	Purpose
Docker	24.0	docker.com	docker --version	Run each service in its own container
Docker Compose	2.20	Included in Docker Desktop	docker compose version	Deploy and coordinate the twenty-five containers
Git	2.40	git-scm.com	git --version	Download the source code from the repository

Source. Own elaboration.

Note. The orchestration tool supports two invocation forms. Current versions use `docker compose`, in two words, since it is a Docker subcommand; earlier versions used `docker-compose`, with a hyphen, since it was a standalone program. This manual always uses the current form. If the machine only recognizes the hyphenated form, the platform must be updated: the project uses the *include* directive, which requires version 2.20 or higher.

Optional Tools

The tools listed below are not necessary for the installation, but they facilitate later review and diagnostic tasks.

Table 9

Complementary support tools

Tool	Situation of use	Function
Visual Studio Code	Review or editing of the code	Editor compatible with the project's four languages
Postman or Insomnia	Manual testing of the programming interface	Sending requests without writing terminal commands

Tool	Situation of use	Function
DBeaver or pgAdmin	Querying the relational database	Graphical client for PostgreSQL
MongoDB Compass	Reviewing the biometric vectors	Graphical client for MongoDB
Mailpit	Testing email sending	Simulated mailbox for development environments

Source. Own elaboration.

Port Availability

The system requires fifteen free ports. When one of them is occupied by another program, the corresponding container does not start.

Table 10

Required ports and frequent conflicts

Port	Service	Exposure	Frequent conflict
8000	Gateway	The whole network	Development servers such as Django, PHP, or Portainer
8090	Web interface	The whole network	Infrequent
8081 to 8089	The nine microservices	Local machine only	Ports 8081 and 8085 are often occupied by other development environments
5432	PostgreSQL	Local machine only	Very frequent: a prior PostgreSQL installation on the machine
27017	MongoDB	Local machine only	A prior MongoDB installation
6379	Redis	Local machine only	A prior Redis installation
9092	Kafka	Local machine only	Infrequent

Source. Own elaboration.

Checking the Availability of a Port

On Windows, using the command interpreter or PowerShell, the following instruction is used, replacing the number with the port to be checked:

netstat -ano | findstr ":5432"

On Linux and macOS the following instruction is used:

lsof -i :5432

Verification. When the command returns no lines, the port is free. When it returns one or more lines, the port is occupied: the program must be identified and stopped, or else the port must be changed in the configuration file, as explained in the Initial Configuration section.

Network Connectivity

During the first installation, the machine downloads container images and dependencies from the internet. If working behind a corporate proxy server or a restrictive firewall, access must be ensured to the following domains:

- hub.docker.com and the docker.io subdomains, for the container base images.
- registry.npmjs.org, for the dependencies of the TypeScript services and of the user interfaces.
- repo.maven.apache.org, for the dependencies of the Java services.
- pypi.org, for the dependencies of the Python service.
- github.com, for downloading the project source code.

Download and Installation

This section covers the installation of the base tools and the download of the project. On completing it, the source code will reside on the machine and the container platform will be in a position to build it.

Installing Docker

Procedure on Windows

1. Go to the address docker.com/products/docker-desktop and download the Windows version.
2. Run the downloaded installer and keep the option to use the Windows Subsystem for Linux instead of Hyper-V selected.

3. Restart the machine when the installation finishes. The restart is mandatory for the subsystem to become active.

4. Open the application from the start menu and wait until the panel indicates that the engine is running.

Procedure on Ubuntu and Debian

```
# 1. Install the container engine and the orchestration plugin
curl -fsSL https://get.docker.com | sudo sh

# 2. Allow use without superuser privileges
sudo usermod -aG docker $USER

# 3. Log out and log back in to apply the group change
```

Procedure on macOS

1. Download the macOS version, selecting the variant corresponding to the machine's processor: Apple Silicon or Intel.

2. Drag the application to the Applications folder and open it.

3. Grant the permissions requested by the operating system.

Verifying the Docker Installation

```
docker --version
docker compose version
docker run hello-world
```

Verification. The first two commands must display a version number. The third must end with a message confirming that the installation works correctly. The appearance of that message certifies that the platform was installed correctly.

Installing Git

Table 11

Git installation procedure by operating system

Operating system	Procedure
Windows	Download the installer from git-scm.com/download/win and run it, accepting the default options
Ubuntu and Debian	Run: sudo apt update && sudo apt install -y git

Operating system	Procedure
macOS	Run: brew install git, or else install the Xcode command line tools

Source. Own elaboration.

git --version

Verification. The command must display version 2.40 or higher.

Obtaining the Project

There are two routes for obtaining the source code (FaceAttend EDU development team, 2026). The first should be used when access to the repository is available.

First Route: Cloning the Repository

```
# 1. Go to the folder where the project will be installed
cd C:\Users\<user>\Documents    # Windows
cd ~/Documents                  # Linux and macOS

# 2. Clone the repository
git clone https://github.com/Jonas7891/ProyectoFaceAttendEDU.git

# 3. Enter the folder containing the code
cd ProyectoFaceAttendEDU
```

Warning. All of the commands in this manual are run from the ProyectoFaceAttendEDU folder, the root of the repository. It is the only folder from which the orchestration tool locates the configuration file and the files of the system's three tiers. Running them from another location produces configuration-not-found errors.

Second Route: Compressed Archive

When the project is received as a compressed archive, proceed as follows:

1. Extract the archive to a path whose name contains no spaces or accented characters, since such paths cause problems in some container platform commands.

2. Open a terminal inside the resulting ProyectoFaceAttendEDU folder.

Verifying the Project Structure

Confirm that the root folder contains the elements represented in Figure 2.

Figure 2

Project folder structure

```
ProyectoFaceAttendEDU/
|
+-- .env.example            configuration template
+-- docker-compose.yml      main orchestrator, entry point
+-- COMPOSE.md              infrastructure reference
|
+-- back-end/               the nine microservices and the gateway
|   +-- 01-ms-identity/        Java 21 and Spring Boot
|   +-- 02-ms-authorization/   Java 21 and Spring Boot
|   +-- 03-ms-academic/        TypeScript and Fastify
|   +-- 04-ms-scheduling/      Java 21 and Spring Boot
|   +-- 05-ms-attendance/      Java 21 and Spring Boot
|   +-- 06-ms-biometric/       Python 3.12 and FastAPI
|   +-- 07-ms-configuration/   TypeScript and Fastify
|   +-- 08-ms-notification/    TypeScript and Fastify
|   +-- 09-ms-quality/         TypeScript and Fastify
|   +-- 99-api-gateway/        Kong OSS 3.6
|
+-- database/               database and the eight migrations
|   +-- database-init/         creation of the eight schemas
|
+-- front-end/
|   +-- Web/                   web application
|   +-- Mobile/                mobile application
|
+-- tools/
    +-- seed-api-test-data/    optional loading of test data
```

Source. Own elaboration.

The check is performed with the command corresponding to the operating system:

```
dir      # Windows
ls -la   # Linux and macOS
```

Verification. The files docker-compose.yml and .env.example must appear, as well as the folders back-end, database, front-end, and tools. The absence of any of them indicates that the download was incomplete, in which case the retrieval procedure must be repeated.

Initial Configuration

All of the system configuration resides in a single file, named .env, located at the root of the ProyectoFaceAttendEDU folder. The orchestration tool reads it automatically and distributes its values across the twenty-five containers.

Creating the Configuration File

The project includes a documented template named .env.example. The first configuration step is to copy it under the name .env:

```
# From the ProyectoFaceAttendEDU folder
copy .env.example .env         # Windows, command interpreter
Copy-Item .env.example .env    # Windows, PowerShell
cp .env.example .env           # Linux and macOS
```

Verification. Using the command `dir .env` on Windows, or `ls -la .env` on Linux and macOS, confirm that the file exists and occupies approximately 3 kilobytes.

Note. The template leaves every password and encryption key empty on purpose, so that no published value can open an installation. The orchestration tool refuses to start while any of them is missing, with a message such as `POSTGRES_PASSWORD is required`. They are generated once, as described in the next subsection; the remaining variables already have working default values.

Generating the Passwords and Keys

The following commands fill in the nineteen empty values of the .env file. They are run once, from the ProyectoFaceAttendEDU folder, in Git Bash on Windows or in the terminal on Linux and macOS; Git Bash includes the openssl tool.

```
# Remove Windows line endings, so that the replacements find every line
sed -i 's/\r$//' .env

# Passwords and tokens: 48 hexadecimal characters each
for v in POSTGRES_PASSWORD IDENTITY_DB_PASSWORD AUTHORIZATION_DB_PASSWORD \
         ACADEMIC_DB_PASSWORD SCHEDULING_DB_PASSWORD ATTENDANCE_DB_PASSWORD \
         CONFIGURATION_DB_PASSWORD QUALITY_DB_PASSWORD NOTIFICATION_DB_PASSWORD \
         MONGO_PASSWORD MINIO_ROOT_PASSWORD EMAIL_INTERNAL_TOKEN \
         BIOMETRIC_LIVENESS_CHALLENGE_SECRET; do
  sed -i "s|^$v=$|$v=$(openssl rand -hex 24)|" .env
done

# Encryption keys: base64 of 32 random bytes each
for v in BIOMETRIC_ENCRYPTION_KEY PII_ENCRYPTION_KEY PII_HASH_KEY; do
  sed -i "s|^$v=$|$v=$(openssl rand -base64 32)|" .env
done

# User names and the key of the document storage
sed -i "s|^MONGO_USER=$|MONGO_USER=faceattend|" .env
sed -i "s|^MINIO_ROOT_USER=$|MINIO_ROOT_USER=faceattend|" .env
sed -i "s|^MINIO_KMS_SECRET_KEY=$|MINIO_KMS_SECRET_KEY=faceattend-key:$(openssl rand -
base64 32)|" .env
```

Verification. The command `docker compose config --quiet` finishes without printing anything. If a value is still missing, it names the variable.

Warning. BIOMETRIC_ENCRYPTION_KEY, PII_ENCRYPTION_KEY, PII_HASH_KEY, and MINIO_KMS_SECRET_KEY encrypt the stored data: without them, the biometric templates, the personal data, and the supporting documents cannot be read. A copy of the .env file must be kept in a safe place.

Warning. The configuration file is never added to the repository. It is deliberately excluded, since it contains credentials. The template values constitute development credentials and not production secrets. For a production environment, all of the passwords must be changed and managed by means of operating system variables or a secrets manager, and never by means of a file belonging to the project.

Environment Variables

The file is organized into thematic blocks. The following tables constitute the complete reference of the available variables.

Table 12

PostgreSQL configuration variables

Variable	Default value	Element it controls
POSTGRES_USER	postgres	Database administrator user
POSTGRES_PASSWORD	Empty, required	Password of that user; generated as described above
POSTGRES_DB	faceattend_db	Name of the database that contains the eight schemas
POSTGRES_PORT	5432	Port on the host machine; must be changed if a prior PostgreSQL installation exists
FACEATTEND_LIQUIBASE_IMAGE	liquibase/liquibase:4.29.0	Image that runs the eight migrations

Source. Own elaboration.

Table 13

MongoDB configuration variables

Variable	Default value	Element it controls
MONGO_USER	Empty, required	User of the document database
MONGO_PASSWORD	Empty, required	Password of that user; generated as described above
MONGO_DB	faceattend_biometric	Database where the facial and fingerprint vectors are stored
MONGO_PORT	27017	Port on the host machine

Source. Own elaboration.

Table 14

Supporting infrastructure variables

Variable	Default value	Element it controls
REDIS_PORT	6379	Port of the store that supports the rate limit
KAFKA_PORT	9092	Port of the domain event broker
KONG_PROXY_PORT	8000	Port through which all requests to the programming interface arrive
KONG_ADMIN_PORT	8001	Reserved: the administration interface is not published to the host

Source. Own elaboration.

Table 15

Network exposure and web interface variables

Variable	Default value	Element it controls
BIND_IP	127.0.0.1	Address to which the internal ports are bound. With the default value they are accessible only from the machine itself; with the value 0.0.0.0 they become visible across the whole local network
FRONTEND_PORT	8090	Port of the web interface
EXPO_PUBLIC_API_URL	Empty	Address of the gateway used by the browser. When empty, the web application uses the host of the page itself with port 8000

Source. Own elaboration.

Warning. The EXPO_PUBLIC_API_URL variable is not read at startup; instead it is baked into the web application bundle at the moment the image is built. Changing it therefore requires rebuilding the interface with the command `docker compose up -d --build`

`frontend-web`; restarting the system is not enough. This circumstance is the most frequent cause of the interface loading but failing to connect to the programming interface.

Table 16

First administrator variables

Variable	Default value	Element it controls
BOOTSTRAP_USERNAME	Empty	User of the first Super Administrator, created as described in Creating the First Administrator
BOOTSTRAP_PASSWORD	Empty	Its current password; the test data tool signs in with this pair

Source. Own elaboration.

Customization Cases

First Case: The Database Port Is Occupied

When the machine has a prior PostgreSQL installation, the port on the host machine must be changed (PostgreSQL Global Development Group, n.d.). The container's internal port remains unchanged, so the rest of the system continues to work without modification:

POSTGRES_PORT=5433

Second Case: Testing from a Mobile Device

The procedure comprises three modifications and one rebuild. First, find out the machine's address on the local network with `ipconfig` on Windows, or `ip addr` on Linux and macOS. Next, edit the variables:

BIND_IP=0.0.0.0

EXPO_PUBLIC_API_URL=http://192.168.1.10:8000

Finally, rebuild the web interface:

docker compose up -d --build frontend-web

Warning. The gateway only accepts requests coming from the origins listed in the KONG_CORS_ORIGINS variable. To serve the application from another address, that address must be added to that variable. Likewise, the value 127.0.0.1 must be restored to the BIND_IP

variable when the test finishes, since the value 0.0.0.0 exposes the databases to the entire local network.

Third Case: The Gateway Port Is Occupied

KONG_PROXY_PORT=8070

EXPO_PUBLIC_API_URL=http://localhost:8070

The web interface must be rebuilt afterwards, for the reason set out in the previous section.

Email Channel

The notification service can send email messages intended for password recovery and account verification. When the mail server variable remains empty, the channel is disabled: the system continues to work normally and merely records a warning in its log files.

To test this functionality in development without having a real mail server, deploy a simulated mailbox:

docker run -d -p 1025:1025 -p 8025:8025 axllent/mailpit

And configure the following variables:

SMTP_HOST=host.docker.internal

SMTP_PORT=1025

SMTP_FROM=FaceAttend EDU <no-reply@faceattend.local>

SMTP_STARTTLS=false

Verification. On going to the address http://localhost:8025 the inbox in which the messages sent by the system are received must be displayed.

Bringing the System Up

With the project downloaded and the configuration file created, the system is deployed with a single command. The database, the tables, and the catalogs are created automatically; user accounts are not, as explained in Creating the First Administrator.

Deploying the System

From the ProyectoFaceAttendEDU folder, with the container platform running, use the following command:

docker compose up -d --build

The command breaks down as follows:

• The verb up creates and starts the defined containers.

• The -d option keeps them running in the background and returns control of the terminal.

• The --build option builds the service images before starting them; it is mandatory on the first run.

Warning. The first run takes a long time, since the platform must download the base images and compile the nine services in three different languages. The estimated time ranges between fifteen and forty minutes, depending on the connection speed and the capacity of the machine. Subsequent runs take between one and three minutes, since they reuse the previously built components. The process must not be interrupted; if it is cancelled, simply run the same command again.

Startup Order

The orchestration tool does not start all of the containers simultaneously, but rather respects a dependency chain intended to prevent a service from attempting to use a resource that does not yet exist.

Table 17

Phases of the system startup order

Phase	Components that start	Condition for advancing
1	Relational database	The database responds correctly to its health check
2	The eight migrations, run consecutively in the order identity, authorization, academic, scheduling, attendance, biometrics, configuration, and notification, and the task that creates the database login of each service	Each migration must finish successfully before the next one starts

Phase	Components that start	Condition for advancing
3	The nine microservices	Each one waits for its own migration and for the database to be in a healthy state
4	Gateway	Waits for the cache store, the messaging broker, and the eight main microservices
5	Web interface	Waits for the gateway to reach the healthy state

Source. Own elaboration.

Note. The practical consequence of this chain is that, when a migration fails, the process stops and the services that depend on it do not start. In the event of any startup difficulty, the migration logs must therefore be reviewed first.

Database and Migrations

There is no need to create the database or to run any structured query file manually: the process is automatic and comprises two stages.

Creation of the Schemas

The first time the relational database starts with its data volume empty, it runs an initialization file that creates the eight schemas corresponding to the system's bounded contexts: identity, authorization, academic, scheduling, attendance, biometrics, configuration, and notification.

Note. The organization into eight schemas, and not into eight independent databases, is due to the fact that each microservice is the exclusive owner of its schema and no service queries another's tables: references between contexts are established by means of identifiers, without physical foreign keys. This is the database-per-context pattern, implemented on a single instance for the purpose of simplifying the deployment.

Creation of Tables and Catalog Data

The eight migration tasks apply, in order, the changes corresponding to each context: first the structure, made up of tables, indexes, and constraints, and then the indispensable catalog data

(Liquibase, n.d.). These are temporary containers that run, finish, and stop; their appearance with a successful exit status in the listing is the expected result.

On subsequent startups, the migration tool detects the previously applied changes and runs only the new ones, which is why it is safe to deploy the system on a day-to-day basis.

Initial System Data

On completion of the migrations, the system contains its catalogs, but no user account: no published password can open an installation.

Table 18

Predefined system roles

Role	Scope of privileges
Super Administrator	Global administration without a campus: activates campuses and manages the whole platform
Administrator	Full access within their campus: users, roles, academic structure, and configuration
Instructor	Teaching: opens and closes class sessions, records attendance, and reviews justifications
Student	Queries their own information and submits justifications

Source. Own elaboration.

The system's base catalogs are also loaded: the permission matrix by role, the academic actor types, the justification types, and the alert types.

Creating the First Administrator

The first Super Administrator is created with a script, run once from the ProyectoFaceAttendEDU folder in Git Bash on Windows or in the terminal on Linux and macOS. The credentials are chosen by the installer and never stored in the repository.

Table 19

Variables of the first administrator script

Variable	Required	Content
BOOTSTRAP_ADMIN_USERNAME	Yes	User name of the administrator
BOOTSTRAP_ADMIN_EMAIL	Yes	Email address of the administrator

Variable	Required	Content
BOOTSTRAP_ADMIN_PASSWORD	Yes	Temporary password of at least twelve characters
BOOTSTRAP_ADMIN_DOCUMENT	No	Identity document; a generated value is used when it is empty

Source. Own elaboration.

```
BOOTSTRAP_ADMIN_USERNAME=admin.local \
BOOTSTRAP_ADMIN_EMAIL=admin@faceattend.local \
BOOTSTRAP_ADMIN_PASSWORD='<at least 12 characters>' \
  ./database/scripts/bootstrap-admin.sh
```

Verification. Signing in at http://localhost:8090 with that user opens the password change screen. The system requires a new password on the first sign-in; until then, the gateway only allows reading and the password change.

Note. The script never runs on its own. The accounts admin.faceattend, instructor.faceattend, and aprendiz.faceattend of earlier versions no longer exist in a new installation, and a migration disables them in databases that still had them.

Loading Test Data

When the system is required to be populated with sample campuses, programs, cohorts, courses, and enrollments — a common situation in demonstrations and in user interface testing — the project includes a tool that creates them by means of calls to the programming interface. It creates one test account per role, listed in Appendix C. Running it requires Node.js version 20 or higher installed on the machine, since it does not use containers:

```
# With the system deployed, all of the services healthy, and the
# first administrator created with its password already changed
cd tools/seed-api-test-data
cp .env.example .env
# In .env, set BOOTSTRAP_USERNAME and BOOTSTRAP_PASSWORD to that administrator
npm run seed
```

Verification. The process finishes with a successful exit code. Running it repeatedly is safe, since the tool detects pre-existing records and continues without duplicating them.

Verifying the Installation

The installation must not be considered finished until this section is completed. Each test checks a different layer of the system, so they must be run in the order proposed: an early failure explains the subsequent ones.

Checklist

The verification is considered complete when the following seven conditions are satisfied:

1. The twenty-five containers appear in the listing: the sixteen services are running and in a healthy state, and the nine one-off tasks have finished.
2. The nine one-off tasks, the eight migrations and the database logins, finished with a successful exit code.
3. Each microservice responds to its health check.
4. The gateway responds on its port.
5. Login returns a valid session identifier.
6. The web interface loads in the browser and allows logging in.
7. The data layer is not reachable from the network that serves the browser.

First Test: Container Status

docker compose ps -a

Verification. A table of twenty-five rows must be displayed. The microservices, the gateway, the web interface, and the databases must appear as running, and those that incorporate a health check must appear in a healthy state. The nine rows corresponding to the one-off tasks must appear with a successful exit status, which certifies that they completed their task.

Warning. When any container appears in a permanent restart state, in an unhealthy state, or with an error exit, the installation is not complete. The logs of that container must be reviewed and the Troubleshooting section consulted before continuing.

To additionally display the published ports, the following command is used:

docker ps --format "{{.Names}} | {{.Status}} | {{.Ports}}"

Second Test: Service Health

Each service exposes a path that confirms its availability. These paths differ according to the implementation technology:

```
# Services implemented in Java
curl http://localhost:8081/api/v1/health      # identity
curl http://localhost:8084/actuator/health    # scheduling

# Services implemented in TypeScript
curl http://localhost:8083/health             # academic
curl http://localhost:8087/health             # configuration

# Through the gateway: 401, because the route requires a session
curl -i http://localhost:8000/api/v1/auth/session
```

Verification. Each service command returns a JavaScript Object Notation response with a positive status, and the gateway answers 401. None of them must return a connection error.

Note. When the command line tool is not available on Windows, the PowerShell instruction `Invoke-RestMethod` may be used instead, or else the address may be opened directly in the browser.

Third Test: Authentication

This is the most important test, since it traverses the complete chain — browser, gateway, identity service, and database — and confirms that the first administrator was created correctly:

```
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"identifier\":\"<administrator-user>\",\"password\":\"<password>\"}"
```

Verification. A response is obtained containing a session identifier, the user identifier, and the active status of the session.

That identifier is then used to check a protected endpoint:

```
curl http://localhost:8000/api/v1/persons \
  -H "Authorization: Bearer <session-identifier>"
```

Note. The system does not use JSON Web Tokens (JWT). On login, the identity service issues an opaque session identifier that is sent with each request by means of the authorization header. The gateway requires that identifier on every non-public route and validates it against the identity service (with a ten-second cache); in addition, each service checks the session and the

permissions again on its own. Consequently, an unauthorized error can come from either the gateway or the destination service.

Fourth Test: Web Interface

1. Open an up-to-date browser.
2. Go to the address http://localhost:8090.
3. Confirm that the system login screen loads.
4. Log in with the administrator user credentials.
5. Confirm access to the main panel corresponding to the administrator role.

Verification. The interface loads, accepts the credentials, and displays the panel. If the page loads but the login does not respond, open the browser developer tools and review the console: connection errors almost always indicate that the programming interface address does not match, a situation addressed in the Troubleshooting section.

The services implemented in Java additionally publish their interactive documentation at the addresses http://localhost:8081/swagger-ui.html and equivalents for ports 8082, 8084, and 8085.

Fifth Test: Network Isolation

This test confirms that the separation into three networks is working: neither the web interface nor the gateway must be able to reach the database.

```
# The following two commands must FAIL
docker exec faceattend-frontend-web nslookup postgres
docker exec faceattend-kong getent hosts postgres

# The following command must WORK
docker exec faceattend-kong getent hosts ms-identity
```

Verification. The first two commands return a name resolution error and the third returns a network address. If the web interface manages to resolve the database name, the networks are not correctly separated and the data layer would be reachable from the layer exposed to the browser.

Once the five tests are passed, the installation is considered complete and verified. The system is in a position to be used in accordance with the user manual, or to be configured with real users in accordance with the administrator manual.

Installing the Mobile Application

The mobile application is not deployed by means of containers; instead it is run in development mode and opened on a physical phone or in an emulator. Its installation is optional with respect to the main system.

Additional Requirements

Table 20

Additional requirements of the mobile application

Requirement	Detail
Node.js	Version 20 or higher, installed on the machine
Expo client application	Installed on the phone, from the corresponding app store
Network	The phone and the machine must be on the same wireless network
Main system	Must be deployed and accessible from the local network, with the exposure variable set to 0.0.0.0

Source. Own elaboration.

Installation Procedure

```
# 1. Find out the machine's address on the local network
ipconfig    # Windows
ip addr     # Linux
ifconfig    # macOS

# 2. Enter the mobile application folder
cd front-end/Mobile

# 3. Install the dependencies (first time only)
npm install

# 4. Start the development server
npx expo start
```

Warning. From the phone, the loopback designation `localhost` refers to the phone itself and not to the computer. It is mandatory to use the machine's address on the local network in the

mobile application configuration, and to keep the exposure variable set to 0.0.0.0 in the main system configuration file.

On starting up, the development tool displays a quick response code in the terminal. Open the client application on the phone and scan that code, after which the application is downloaded to the device and runs.

Verification. The application displays the login screen and allows logging in with the student user credentials. If the screen loads but login fails, the problem lies in the programming interface address or in the machine's firewall, which may be blocking the gateway port.

Day-to-Day Operation

Frequently Used Commands

Table 21

Frequent operation commands

Command	Effect
docker compose up -d	Starts the system reusing the previously built images
docker compose up -d --build	Rebuilds the images before starting; used after modifying the code
docker compose ps	Displays the status and health of each container
docker compose logs -f ms-identity	Displays a service's logs in real time
docker compose logs --tail=100 kong-gateway	Displays the last one hundred log lines of a service
docker compose restart ms-academic	Restarts an individual service
docker compose config	Validates the configuration without deploying the system
docker compose down	Stops and removes the containers, preserving the data
docker stats	Displays the processor and memory consumption of each container

Source. Own elaboration.

Stopping and Restarting the System

```
# Stop the system preserving all of the data
docker compose down

# Start it again
docker compose up -d
```

Note. Stopping the system removes the containers but not the data volumes: the database, the biometric vectors, and the events remain intact. On deploying it again, the system continues in the state in which it was left.

Updating to a New Version

```
# 1. Bring in the changes from the repository
git pull origin develop

# 2. Check whether the configuration template added new variables
#    and, if so, add them to the local configuration file

# 3. Rebuild and start; the pending migrations are applied by themselves
docker compose up -d --build
```

Uninstallation

Warning. The first command in the following block irreversibly deletes all of the system's data: users, attendance records, justifications, and biometric vectors. The operation cannot be undone, so it must be run only when starting from scratch is intended.

```
# Removes containers and data volumes
docker compose down -v

# Additionally removes the built images
docker compose down -v --rmi all

# Finally, delete the project folder if it is no longer needed
```

Troubleshooting

This section gathers the difficulties that occur most frequently, ordered according to the stage of the procedure at which they usually appear. Each entry sets out the symptom, the probable cause, and the instructions leading to its resolution.

Initial Diagnosis

In the event of any failure, run the following three commands, which in most cases identify the cause:

```
docker compose ps                       # which container is failing
docker compose logs --tail=50 <name>    # what that container reports
docker compose config                   # whether the configuration is valid
```

Problems During Installation

Table 22

Frequent problems during installation

Symptom	Probable cause	Solution
The system does not recognize the docker command	The platform is not installed or does not appear in the system path variable	Reinstall the platform and restart the machine. On Linux, log out and log back in after adding the user to the corresponding group
It is not possible to connect to the container service	The service is not running	Open the desktop application and wait until it reports that the engine is active. On Linux, start the service via the system manager
No configuration file is provided	The command is being run from a folder other than the root of the code	Move to the folder that contains the orchestration file
The build fails while downloading dependencies	No connection, or a corporate proxy server is blocking the repositories	Verify the connection and access to the domains listed in the Prerequisites section, and retry the build without cache
No space left on device	Disk saturated with unused images and containers	Prune the unused resources and confirm that at least 20 GB remain free

Source. Own elaboration.

Problems When Deploying the System

The Port Address Is Already Allocated

The cause is that another program on the machine is using that port. The most frequent case corresponds to a prior PostgreSQL installation occupying port 5432.

```
# 1. Identify the process occupying the port
netstat -ano | findstr ":5432"   # Windows
lsof -i :5432                    # Linux and macOS

# 2a. Stop that process
taskkill /PID <identifier> /F    # Windows
kill -9 <identifier>             # Linux and macOS

# 2b. Or else change the port in the configuration file
docker compose up -d
```

A Service Declares an Undefined Dependency

The error occurs when the deployment is run from the services folder instead of the project root. That file, on its own, does not define the database, which resides in another folder.

The solution is always to run the deployment from the root. This is a deliberate design error, intended to make evident which is the correct entry point.

A Container Remains in an Endless Restart Loop

docker compose logs --tail=100 <service-name>

The usual causes include that the database was not yet available, a situation that resolves itself after a few retries; that an environment variable is missing; or that the service cannot resolve the name of another container. If after five minutes the container is still restarting, the complete system must be restarted.

The Web Interface Shows an Unhealthy Status

The known cause is that the health check must use the numeric loopback address and not its name, since inside the container that name also resolves to the sixth-version internet protocol address, on which the web server is not listening, so the connection is refused. The project file already incorporates the fix; if the symptom appears, the web interface must be rebuilt.

Problems with the Migrations

A Migration Ends with an Error

The migration chain stops and no subsequent service starts. Identify which one failed and for what reason:

```
# 1. Identify the failed migration
docker compose ps | findstr liquibase   # Windows
docker compose ps | grep liquibase      # Linux and macOS

# 2. Check the reason for the failure
docker compose logs identity-liquibase

# 3. After correcting, re-run only that migration
docker compose up identity-liquibase

# 4. The chain continues on the next startup
docker compose up -d
```

Warning. As a last resort, when the database is left in an inconsistent state and this is a test environment, it may be reset from scratch by deleting the volumes. That operation erases all of the data and must never be run on an installation containing real information.

The System Reports That a Table or a Schema Does Not Exist

The cause is that the migrations were not run or were left incomplete. Verify that the eight tasks finished with a successful exit code and review the logs of the first one that did not.

Connection and Usage Problems

Table 23

Frequent connection and usage problems

Symptom	Probable cause	Solution
Blank screen at the web interface address	The interface has not yet finished starting, or its build failed	Wait one minute and reload. If it persists, check the web interface logs
The interface loads but does not connect to the programming interface	The configured address does not match the real one; that variable is baked in during the build	Correct the value in the configuration file and rebuild the web interface
The gateway responds with a gateway error	The destination microservice has not yet reached the healthy state	Identify the pending service and review its logs until it reaches that state
The gateway reports that no route matches	The requested route is not declared in the gateway configuration	Check the route in Appendix B. Valid routes begin with the programming interface version prefix
All calls return an unauthorized error	The session header is missing, the session expired, or the user has no roles assigned	Obtain a new identifier via the login endpoint and send it in the authorization header of each request
Calls return a forbidden access error	The session is valid but the role lacks the required permission	Verify the roles assigned to the user via the corresponding endpoint
The browser reports a cross-origin policy error	Access is being made from an origin not authorized in the gateway	Add that origin to the list of allowed origins in the gateway configuration
Email messages are not received	The mail server variable remains empty, so the channel is disabled	This is the default behavior. To enable it, configure the channel as described in the Initial Configuration section

Source. Own elaboration.

Performance Problems

Table 24

Frequent performance problems

Symptom	Cause and solution
The machine becomes considerably slow after deployment	The sixteen running containers, four of them with a Java virtual machine, consume a noticeable amount of memory. Close the other applications and allocate at least 6 GB of memory to the container platform. Consumption per container can be examined with the statistics command
The build takes more than an hour	This is normal behavior on the first run with a slow connection, since approximately 5 GB are downloaded. Subsequent runs are considerably faster. The process must not be interrupted
The containers stop by themselves	The platform ran out of the allocated memory. Increase it or deploy only the required services

Source. Own elaboration.

Clean Restart Procedure

When the previous alternatives are exhausted and the system still does not work, the following procedure restores the state of a fresh installation:

```
# 1. Stop and remove everything, including the data
docker compose down -v

# 2. Prune the unused resources
docker system prune -a

# 3. Verify that the configuration file exists and is correct

# 4. Rebuild from scratch
docker compose up -d --build
```

Warning. The procedure deletes all of the system's data, so it must be used exclusively in development or assessment environments.

Incident Escalation

When a difficulty is not resolved by the previous procedures, it must be reported using the format in Appendix D. The report must attach the output of the three initial diagnostic commands; without that evidence, remote diagnosis is practically unfeasible.

Installation Without Containers

Warning. This section is optional and is not recommended for an ordinary installation. It is relevant only when the code of a service is to be modified and debugged from the development

environment. To install and use the system, the container-based procedure is faster and considerably less error-prone.

Tools Required by Programming Language

Each service requires its own runtime environment installed on the machine.

Table 25

Tools required by programming language

Language	Version	Affected services	Source	Verification
Java	21 LTS	Identity, authorization, scheduling, and attendance	adoptium.net	java --version
Maven	3.9	The same four services	maven.apache.org	mvn --version
Node.js	24	Academic, configuration, notification, quality, and both interfaces	nodejs.org	node --version
Python	3.12	Biometrics	python.org	python --version

Source. Own elaboration.

Recommended Approach: Hybrid Mode

The usual practice is not to dispense with containers entirely, but to deploy the whole infrastructure with them — databases, messaging, gateway, and the services that are not being modified — and to run outside them only the service being worked on:

```
# 1. Deploy only the data layer and the migrations
docker compose up -d database

# 2. Run a service from its folder, according to the language
cd back-end/01-ms-identity && mvn spring-boot:run
cd back-end/03-ms-academic && npm install && npm run dev
cd back-end/06-ms-biometric && uvicorn main:app --reload
cd back-end/08-ms-notification && go run ./cmd/server
```

Note. The services read the same configuration file used by the containers. On starting up outside them, each service loads the configuration from the project root by means of the mechanism native to its language, so there is no need to duplicate the configuration.

Web Interface in Development Mode

```
cd front-end/Web
npm install
npx expo start --web
```

The interface starts with automatic reloading when changes are saved.

Security Recommendations

The following guidance applies from the moment the installation stops being an isolated test environment and becomes accessible to third parties.

Credentials

- Never run the test data tool on an installation accessible to third parties, and deactivate any test account that exists there.
- Replace the default credentials of the relational and document databases with values specific to the installation.
- Do not share the configuration file over messaging channels and do not attach it to incident reports without first removing the passwords.
- Keep the configuration file outside the repository; its exclusion is already declared in the project and must not be reverted.

Network Exposure

- Keep the exposure variable set to the loopback address, so that the databases and the microservices are unreachable from the local network.
- Restore that value at the end of any test that required opening the ports to the network.
- Publish only the two ports intended for the outside: that of the web interface and that of the gateway.
- Verify the isolation between networks by means of the fifth test of the Verifying the Installation section after every change to the infrastructure.

Preservation of Information

- Run the stop command without the volume removal option in day-to-day operation, since that option destroys all of the data irreversibly.
- Generate a backup before any update that incorporates new migrations, in accordance with the procedure described in the administrator manual.
- Reserve the clean restart procedure for development or assessment environments.

Handover to the System Administrator

On completing the installation, the installer hands over to the administrator the initial credentials, the warning about their mandatory modification, and the evidence that the verification protocol was carried out in full. From that moment on, responsibility for the system rests with the administrator and is governed by their own manual.

Glossary

The following table defines the technical terms used in this manual, in the specific context of the project.

Table 26

Glossary of system terms

Term	Definition
Container	Package comprising a program and everything necessary for it to work: base system, libraries, and configuration. It runs isolated from the rest of the machine, so it behaves identically on any machine
Image	Read-only template from which containers are created. Building an image is equivalent to compiling the service and packaging it; starting a container is equivalent to running a copy of that image
Docker	Platform that builds and runs containers
Docker Compose	Tool that deploys several related containers with a single command, respecting the dependency order declared in its configuration file
Volume	Disk space managed by the container platform, where containers store the data that must survive their removal
Microservice	Independent service, responsible for a single portion of the business, with its own database and deployment cycle
Gateway	Component that receives all of the external requests and directs them to the corresponding microservice. It also applies the cross-origin policy and the rate limit
Kong	Open-source gateway used in the project. It is configured declaratively, without a database of its own
Endpoint	Specific address of the programming interface that serves a given operation
Schema	Logical subdivision within a relational database. The system uses a single database divided into eight schemas, one per context
Migration	Versioned change to the database structure. Migrations are applied in order and are recorded, so that they are never run twice
Liquibase	Tool that runs the migrations. In the project it comprises eight tasks, one per schema, which are run automatically during the deployment

Term	Definition
Seed data	Minimum records the system requires in order to start: roles, permissions, justification types, and the initial administrator user
Environment variable	Configuration value defined outside the code, which the program reads at startup. It allows ports or credentials to be changed without altering the source code
Port	Number that identifies a communication channel on a machine. Two programs cannot use the same port simultaneously
Loopback address	Address that designates the machine itself. A service bound to it is accessible only from that machine
Health check	Periodic verification that the container platform runs against a container in order to determine whether it is working correctly
Cross-origin policy	Browser security mechanism that blocks requests directed at a domain other than that of the page, unless expressly authorized by that domain
Opaque session	Identifier with no human-readable content that the server issues on login and that the client sends with each request
Role-based access control	Model in which permissions are assigned to roles and roles are assigned to users
Kafka	Messaging system that allows microservices to communicate asynchronously by means of events, without calling each other directly
Redis	In-memory data store. In the project it holds the rate limit counters
MongoDB	Document-oriented database. In the project it stores the biometric vectors
Expo	Platform that allows the web application and the mobile application to be built from the same code
nginx	Web server that delivers the previously built interface files to the browser
Windows Subsystem for Linux	Windows layer that allows Linux to be run. The container platform uses it to run the containers

Source. Own elaboration.

References

Docker Inc. (n.d.-a). *Docker docs*. Retrieved October 6, 2026, from https://docs.docker.com/

Docker Inc. (n.d.-b). *Docker Compose documentation*. Retrieved October 6, 2026, from https://docs.docker.com/compose/

FaceAttend EDU development team. (2026). *FaceAttend EDU* (Version 1.0) [Software]. GitHub. https://github.com/Jonas7891/ProyectoFaceAttendEDU

Git. (n.d.). *Git documentation*. Retrieved October 6, 2026, from https://git-scm.com/doc

Liquibase. (n.d.). *Liquibase documentation*. Retrieved October 6, 2026, from https://docs.liquibase.com/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 17 documentation*. Retrieved October 6, 2026, from https://www.postgresql.org/docs/17/

Appendix A

Complete Port Map

This appendix gathers in one place all of the ports used by the system, their degree of exposure, and the address through which each one is accessed.

Table 27

Complete system port map

Port	Service	Exposure	Access address
8090	Web interface	The whole network	http://localhost:8090
8000	Gateway	The whole network	http://localhost:8000
8081	Identity	Local	http://localhost:8081/swagger-ui.html
8082	Authorization	Local	http://localhost:8082/swagger-ui.html
8083	Academic	Local	http://localhost:8083/health
8084	Scheduling	Local	http://localhost:8084/actuator/health
8085	Attendance	Local	http://localhost:8085/actuator/health
8086	Biometrics	Local	http://localhost:8086/docs
8087	Configuration	Local	http://localhost:8087/health
8088	Notifications	Local	http://localhost:8088
8089	Quality	Local	http://localhost:8089
5432	PostgreSQL	Local	localhost:5432
27017	MongoDB	Local	localhost:27017
6379	Redis	Local	localhost:6379
9092	Kafka	Local	localhost:9092

Source. Own elaboration.

Note. The exposure designated as local corresponds to the value of the BIND_IP variable, whose default value is 127.0.0.1.

Appendix B

Main Routes of the Programming Interface

All of the routes listed below pass through the gateway and require the authorization header with the session identifier, except for the public authentication endpoints (login and its verification, refresh, logout and password recovery) and the campus sign-up.

Table 28

API routes by destination service

Destination service	Routes
Identity	/api/v1/auth, /api/v1/persons, /api/v1/users, /api/v1/sessions, /api/v1/cities, /api/v1/password-policies
Authorization	/api/v1/roles, /api/v1/permissions, /api/v1/auth/evaluate, /api/v1/users/{id}/roles
Academic	/api/v1/schools, /api/v1/programs, /api/v1/academic-periods, /api/v1/cohorts, /api/v1/courses, /api/v1/academic-actors, /api/v1/enrollments
Scheduling	/api/v1/environments, /api/v1/schedule-blocks, /api/v1/class-sessions
Attendance	/api/v1/attendance-records, /api/v1/justifications, /api/v1/justification-types, /api/v1/supporting-documents
Biometrics	/api/v1/biometric
Configuration	/api/v1/academic-configurations, /api/v1/security-configurations, /api/v1/biometric-update-cases
Notifications	/api/v1/alert-types, /api/v1/alerts
Quality	/api/v1/quality, /health/quality

Source. Own elaboration.

Appendix C

Test Accounts by Role

Warning. These accounts exist only after running the test data tool, and only for local installations. Their passwords are the values of the tool's .env file, copied from a public template: the tool must never be run against an installation accessible to third parties.

Table 29

Test accounts by role

User	Role	Created by	Password
The value of BOOTSTRAP_ADMIN_USERNAME	Super Administrator	bootstrap-admin.sh	The one chosen when running the script, changed on the first sign-in
carolina.mendoza	Super Administrator	Test data tool	SEED_PASSWORD in tools/seed-api-test-data/.env
admin.ict and admin.smp	Administrator	Test data tool	SCHOOL_ADMIN_PASSWORD in the same file
carlos.restrepo	Instructor	Test data tool	INSTRUCTOR_PASSWORD in the same file
valentina.rios	Student	Test data tool	STUDENT_PASSWORD in the same file

Source. Own elaboration.

Note. The tool also creates eighty instructor accounts, named name.surname, which sign in with TEACHER_PASSWORD.

Appendix D

Incident Report Format

When a difficulty is not resolved by means of the Troubleshooting section, it must be reported by recording the information listed below. In the absence of this data, remote diagnosis is practically unfeasible.

```
FACEATTEND EDU INSTALLATION - INCIDENT REPORT

1. Environment details
   Operating system and version  : _____________________________
   Machine memory                : _____________________________
   Platform version              : (output of docker --version)
   Orchestrator version          : (output of docker compose version)

2. Point in the procedure where it occurred
   Manual section                : _____________________________
   Command executed              : _____________________________

3. Observed behavior
   Expected result               : _____________________________
   Obtained result               : _____________________________
   Complete error message        : _____________________________

4. Attached evidence
```

```
   ( ) Output of docker compose ps
   ( ) Output of docker compose logs for the affected service
   ( ) Screenshot of the error
   ( ) Contents of the configuration file, without the passwords

5. Actions previously attempted
   ____________________________________________________________
```

Appendix E

Summary of the Installation Procedure

This appendix gathers the complete sequence for whoever already knows the procedure and needs only the succession of commands.

```
# 1. Obtain the project
git clone https://github.com/Jonas7891/ProyectoFaceAttendEDU.git

# 2. Enter the folder containing the code
cd ProyectoFaceAttendEDU

# 3. Create the configuration file and generate its passwords and keys
cp .env.example .env
# (commands of the Generating the Passwords and Keys section)

# 4. Deploy the system (15 to 40 minutes the first time)
docker compose up -d --build

# 5. Verify the status of the containers
docker compose ps -a

# 6. Create the first administrator
BOOTSTRAP_ADMIN_USERNAME=admin.local BOOTSTRAP_ADMIN_EMAIL=admin@faceattend.local \
BOOTSTRAP_ADMIN_PASSWORD='<at least 12 characters>' ./database/scripts/bootstrap-
admin.sh

# 7. Open the web interface and sign in with that administrator
# http://localhost:8090
```

Verification. The procedure is considered finished only when the five tests of the Verifying the Installation section are passed.